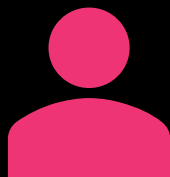

FALL 2026

AARON LIN |

**OLIVER COOPER | MAYRA GONZALEZ MARTINEZ |
KARANVEER SINGH**



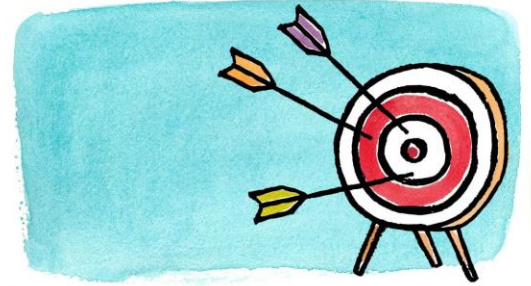
**CSC 370
KICKOFF**



**MOVIE RATING
DATABASE**

KICK-OFF GOALS

1. **Conceptual Modeling (ERD):** Design a complete Entity-Relationship diagram capturing all core requirements for a movie rating system.
2. **Relational schema and BCNF Normalization:** Map the conceptual ERD into a relational schema normalized up to Boyce-Codd Normal Form (BCNF).
3. **SQL DDL Implementation:** Implement the normalized relational schema in SQL using formal Data Definition Language (DDL).



ERD

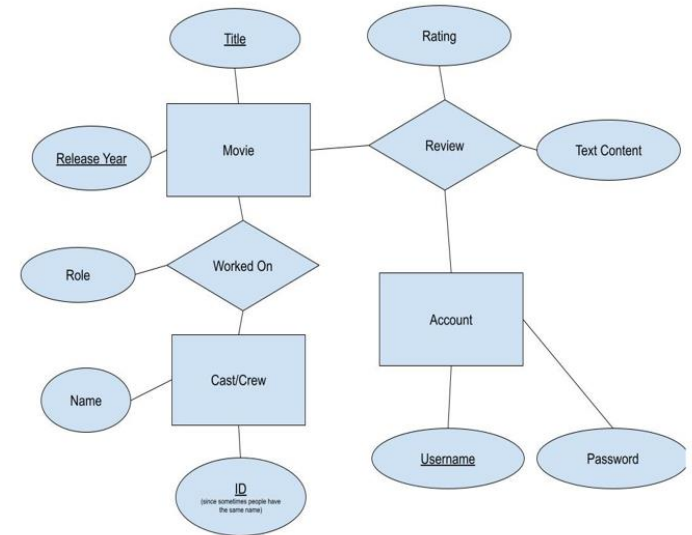
Identifier :The idea behind the composite key on Movie is that titles and release years can each repeat, so only the two together identify a movie.

Identifier : Since names can often be the same, we also use an ID as the primary key for Cast/Crew instead of name.

Relationship Attributes : Review and Worked On are relationships that connect two entities and store their own attributes: a rating and text for Review, and a role for Worked On.

Multiplicity : Both relationships are many-to-many. A movie can have many cast and crew members, and a person can work on many movies. Likewise, a user can review many movies, and a movie can receive many reviews. Each user can review a given movie only once.

Separate Entity :Account is kept separate from Review, so a user can exist without having written any reviews.



ENTITIES → TABLES

- **Mapping to DDL** : Each entity from the ERD becomes its own table using SQL Data Definition Language
- **Movie Entity** : The movie table uses a composite primary key of title and release year, since both can repeat on their own.
- **Cast_Crew** : The cast_crew table uses id as its primary key instead of name, because different people can share the same name.
- **Account** : The account table uses username as its primary key, so every account is unique.
- **Enforcing Valid Data** : Each table uses defined data types and not null to prevent invalid or incomplete tuples.

```
-- 1. Entity 'movie'
CREATE TABLE movie (
    title VARCHAR(255) NOT NULL,
    release_year INT NOT NULL,
    PRIMARY KEY (title, release_year)
);
```

```
-- 2. Entity 'cast'
CREATE TABLE cast_crew (
    id INT PRIMARY KEY NOT NULL,
    name VARCHAR(255) NOT NULL
);
```

```
-- 3. Entity 'account'
CREATE TABLE account (
    username VARCHAR(50) PRIMARY KEY NOT NULL,
    password VARCHAR(255) NOT NULL
);
```

RELATIONSHIP → TABLE: WORKED ON

- Since a movie can have many cast members and any person can work on many movies at the same time. We implemented a many-many relationship between them
- We use primary key of four columns to uniquely identify each tuple.
- We included “role” in primary key to avoid errors as a single person can be both “actor” and “director” on the same movie.

```
-- 4. Relation 'worked on' (connects movie and cast, adds role attribute)
CREATE TABLE worked_on (
    role VARCHAR(100) NOT NULL,
    title VARCHAR(255) NOT NULL,
    release_year INT NOT NULL,
    cast_id INT NOT NULL,
    PRIMARY KEY (title, release_year, cast_id, role),
    FOREIGN KEY (title, release_year) REFERENCES movie(title, release_year),
    FOREIGN KEY (cast_id) REFERENCES cast_crew(id)
);
```

RELATIONSHIP → TABLE: REVIEW

- This table links a specific person's account to specific movie review.
- Added a check constraint that keeps the rating between 1.0 and 10.0 and prevents invalid data.
- Combined “username”, “title” and “release_year” into a primary key to ensure each user can review the specific movie once.

```
-- 5. Relation 'review'
CREATE TABLE review (
    rating DECIMAL(3, 1) NOT NULL,
    CONSTRAINT chk_rating CHECK (rating >= 1.0 AND rating <= 10.0),
    text_content TEXT,
    username VARCHAR(50) NOT NULL,
    title VARCHAR(255) NOT NULL,
    release_year INT NOT NULL,
    PRIMARY KEY (username, title, release_year),
    FOREIGN KEY (username) REFERENCES account(username),
    FOREIGN KEY (title, release_year) REFERENCES movie(title, release_year)
);
```

NORMALIZATION: BCNF

Table	Primary key	Non-Trivial FDs (X -> Y)
account	username	username -> password
cast_crew	id	id -> name
movie	(title, release_year)	none (composite key)
worked_on	(title, release_year, cast_id, role)	none (all-key relation)
review	(username, title, release_year)	(username, title, release_year) -> (rating, text_content)

- In every relation, all determinants (X) are candidate/superkeys.
- No partial dependencies or transitive dependencies exist across the schema.
- Fully satisfies Boyce-Codd Normal Form (BCNF) without data redundancy.

DEMONSTRATION VIDEO

PLAN FOR NEXT SPLIT



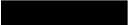
Import Data



Advanced Query



Statistics like top rated



SPRINT 1 GOALS

1. **Constraints & Referential Integrity Actions:** Implement ON DELETE CASCADE on foreign keys (reviews & credits).
2. **Advanced Conceptual Modeling (Subclasses & Inheritance):** Extend cast_crew into role-specific subclasses (Director / Actor).
3. **Minimal Bases & 4NF Normalization:** Incorporate multi-valued movie attributes (e.g., Genres, Languages).